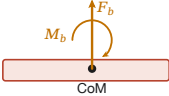
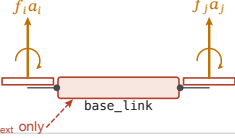


One contract, two implementations `sim/base.py` Protocols; the control loop, runner and recorder are identical for both

VehicleBackend one vehicle; SI units; world Z up; CoM state; wrenches about the CoM reset(initial_position_w=None) place the vehicle, reset motors and sampled k_f, τ read_state(t) → VehicleState CoM position, wxyz body→world, v_w, Ω_b mass_properties() → (m, J, c) whole-vehicle mass, full CoM inertia, CoM in body axes allocation_matrix_b 6×4: rotor thrusts → body wrench about the CoM motor_parameters() thrust_to_motor_speeds(f) apply_motor_speeds(n, w_{ext}) telemetry() sampled $k_{f,i}, [\underline{f}, \bar{f}], [n_{min}, n_{max}]$, current RPS $n_i = \sqrt{f_i/k_{f,i}}$ within running limits; 0 stays 0 one motor step + submit forces; exactly once per tick RPS, commanded / applied thrust, motor wrench, sources	
SimulationDriver step() advance exactly one Δt is_running() is_paused() idle() keep UI alive while paused optional: ground_start_position, assert_control_compatible, describe	
implements	
 SyntheticBackend + SyntheticDriver sim/synthetic.py CPU, NumPy only · local & CI · ≈12 s for the 20 s helix	
Model source	SyntheticPlantParameters: deliberately asymmetric test values, not ARL data. $m = 1.24$ kg; full J with products of inertia; CoM offset $c = (6, -4, 2)$ mm; $k_f = (1.1, 1.3, 1.6, 1.8) \times 10^{-5}$ N/rps ² ; $\tau_{inc} = 50$ –80 ms, $\tau_{dec} = 4.5$ –6 ms; $f \in [0.1, 10]$ N; $k_r = 0.07$ m.
Allocation matrix	$A_i = [a_i; (r_i - c) \times a_i + \sigma_i k_r a_i]$ from the synthetic rotor positions (± 0.1 m) and c ; $\sigma = (-1, +1, +1, -1)$.
Motor model	TranscribedThruster: NumPy transcription of the upstream Thruster integrators (contract tests compare it with the Isaac Lab source when available), driven through the shared NativeRpsActuator.
Force application	Submitted wrench $w = A f_{actual} + w_{ext}$ acts on one rigid body at its CoM (body axes). 
Rigid-body step	RK4 on (p, v, q, Ω) : $\dot{v} = g + RF_b/m$, $\dot{q} = \frac{1}{2} q \otimes (0, \Omega)$, $\dot{\Omega} = J^{-1}(M - \Omega \times J \Omega)$. Floor: vertical inelastic impulse at the CoM only (no friction, contact torque, drag).
State readout	State is stored directly as CoM quantities: position, v_w , wxyz quaternion, Ω_b .
Driver & app	SyntheticDriver: always running, never paused. run_simulation() composes loop + runner + metrics/figure hooks.
Only in this backend	Golden closed-loop regression (tests/data/golden_synthetic.npz); parallel sweep workers.
implements	
 ARLBackend + IsaacDriver sim/isaac/ Isaac Sim 6.1 + Isaac Lab 3.0 + PhysX · GPU server uc1	
ARL-Robot-1 USD: base_link + 4 rotor bodies on fixed joints. PhysX-resolved per-body mass, CoM and principal inertia are rotated and shifted (parallel-axis theorem) into the whole-vehicle m, c, J . Rotor origins from link poses. k_f, τ sampled at reset from the configured ranges.	
Same formula from the actual link geometry and PhysX CoM; the upstream reference matrix (± 0.13 m) is kept only as a logged difference.	
Upstream isaacLab_contrib Thruster wrapped by the same NativeRpsActuator: RPS stored directly, τ_{inc}/τ_{dec} by sign, mixing $1/(\Delta t + \tau)$, rate clamp, RK4, $f = k_f n^2$.	
$f_i a_i$ and reaction torque $\sigma_i k_r f_i a_i$ at each rotor body's origin, all bodies in one PhysX call per tick: apply_forces_and_torques_at_position base_link receives only w_{ext} ; no controller wrench is ever applied to the root. 	
PhysX articulation: gravity, fixed joints, collision shapes vs the ground plane, native damping (0 in the ARL config). IsaacDriver.step() = sim.step(render=False) + robot.update(dt).	
Root-link pose (xyzw) converted at the boundary: $p = p_{root} + Rc$, $v = v_{root} + \omega \times Rc$, $q: xyzw \rightarrow wxyz$, $\Omega_b = R^T \omega_w$.	
app.py: AppLauncher (Kit), scene build, WebRTC UI pump, --wait-for-start. Paused ⇨ not sim.is_playing(); idle() renders. Safety checks: free base, dynamic bodies, rank-4 A , one apply per tick.	
Camera rig + synchronized MP4; path overlay prims; PhysX solver acceleration; USD layer SHA-256; inspect.	